

Beta 3 · Build Planner

Knowledge Graph + Multi-Agent Orchestration

Beta 3 introduces the intelligence layer. A semantic knowledge graph connects machines, SKUs, suppliers, and quality data. Agents share context through the graph. A multi-agent orchestration layer allows agents to hand off to each other and collaborate on cross-functional exceptions.

BUILDS ON	EFFORT	NEW STACK	KEY UNLOCK
Beta 2 (live API)	8–10 days	Neo4j + LangGraph	Cross-agent reasoning

01 OBJECTIVE

Beta 3 makes the system **contextually aware**. When a supplier delivery is delayed, the Supply Agent queries the knowledge graph: which SKUs are affected? Which production orders depend on those SKUs? Which machines will go idle? Which customer orders are at risk? The answer emerges from graph traversal — not hard-coded logic. Agents can now hand off to each other through a LangGraph workflow.

New capabilities in Beta 3
• Neo4j knowledge graph: entities (Machine, SKU, Supplier, Order, WorkCenter) + relationships • Graph-enriched context: every agent prompt includes relevant subgraph as JSON • LangGraph multi-agent workflow: Supply Agent can trigger Quality Agent and vice versa • Impact propagation: one disruption automatically surfaces downstream effects • Graph visualizer panel: interactive node-link diagram in the dashboard • Explainability trace: UI shows the agent's reasoning chain and graph path used

02 KNOWLEDGE GRAPH SCHEMA

Node types and relationship types to model in Neo4j:

Node Type	Key Properties	Relationships
Machine	id, name, plant, oeeTarget, currentOEE, status	PRODUCES → SKU, LOCATED_IN → WorkCenter
SKU	id, name, category, safetyStock, leadTime	REQUIRES → Component, SUPPLIED_BY → Supplier
Supplier	id, name, country, riskScore, leadTime	SUPPLIES → SKU, SERVES → Plant

ProductionOrder	id, sku, qty, dueDate, status	USES → Machine, FULFILLS → CustomerOrder
CustomerOrder	id, customer, deliveryDate, priority	CONTAINS → SKU, AT_RISK_FROM → Exception
QualityEvent	id, type, batch, defectRate, disposition	AFFECTS → SKU, TRIGGERED_BY → Machine
Exception	id, category, severity, timestamp, status	IMPACTS → SKU, INVOLVES → Supplier

03 FILE STRUCTURE ADDITIONS

Add these to the Beta 2 monorepo:

File / Folder	Extension	Purpose
backend/graph/	(folder)	Knowledge graph layer
■ ■■■ neo4j_client.py	.py	Neo4j driver connection + query helpers
■ ■■■ schema.py	.py	Node/relationship class definitions
■ ■■■ seed_data.py	.py	Script to populate graph with sample data
■ ■■■ queries.py	.py	Cypher query library (get_supplier_risk, get_sku_impact, etc.)
■ ■■■ enricher.py	.py	Takes exception dict → queries graph → returns enriched context
backend/orchestration/	(folder)	Multi-agent LangGraph workflow
■ ■■■ graph_workflow.py	.py	LangGraph StateGraph definition
■ ■■■ state.py	.py	Shared agent state schema (TypedDict)
■ ■■■ nodes.py	.py	Each agent as a LangGraph node function
■ ■■■ router.py	.py	Conditional edge logic (which agent next?)
frontend/src/components/	(additions)	
■ ■■■ KnowledgeGraphPanel.jsx	.jsx	Interactive graph vis using react-force-graph
■ ■■■ AgentTrace.jsx	.jsx	Reasoning chain timeline for each agent action
■ ■■■ ImpactMap.jsx	.jsx	Visual cascade: exception → affected nodes
docker-compose.yml	.yml	Spins up FastAPI + Neo4j together
.env	.env	NEO4J_URI, NEO4J_USER, NEO4J_PASSWORD — gitignored

04 BUILD ORDER

01	Days 1–2 — Neo4j setup + schema Install Neo4j Desktop or use AuraDB free tier. Build schema.py and seed_data.py. Seed ~50 nodes across all 7 types. Verify in Neo4j Browser.
02	Days 3–4 — Graph queries + enricher Write Cypher queries for: get_supplier_impact(supplier_id), get_sku_dependencies(sku_id), get_at_risk_orders(exception_id). Build enricher.py to call these for any incoming exception.

03	Days 5–6 — LangGraph multi-agent workflow Define StateGraph. Each agent becomes a node. Router logic: if supply exception touches quality data → trigger quality agent node next. Test with 3 exception types.
04	Day 7 — Frontend graph visualizer Add react-force-graph. Fetch subgraph from new /api/graph/{exception_id} endpoint. Render interactive node-link diagram in KnowledgeGraphPanel.jsx.
05	Day 8 — Agent trace + impact map Build AgentTrace component: show which agents ran, in which order, what each decided. Build ImpactMap showing cascade from exception to affected orders.
06	Days 9–10 — Integration + demo polish End-to-end test: fire exception → enricher queries graph → multi-agent workflow → UI shows trace + impact. Fix any broken edges. Record new demo video.

05 NEW DEPENDENCIES

Backend additions:

```
neo4j (Python driver)
langgraph
langchain
langchain-openai
langchain-anthropic
typing-extensions
```

Frontend additions:

```
react-force-graph
d3
```

Infrastructure:

```
Docker + docker-compose (for Neo4j container)
```